

The AMC Engine — Mathematical Specification

The AMC Engine — Mathematical Specification

An implementation-grade specification of the American Monte-Carlo (AMC) pricing engine: how a compiled `.amc` product, together with a factor model, is turned into a grid of present-value (PV) paths and the exposure profiles derived from them. This document is self-contained: given the DSL reference (for the product side) and this spec (for the engine side), the entire pipeline that produces the PV grid can be re-implemented from scratch.

1. Overview — what the engine computes

The DSL front end compiles a `.amc` script into a **Product**: an ordered list of dated **events** (`fix`, `pay`, `decide/american`, `pay_undisc`, ...) plus a **state** specification (per-path variables, some flagged as regressors). The engine's job is to take that **Product** and a **Model** (a Monte-Carlo factor simulator under the risk-neutral measure Q) and produce:

1. a scalar **price** `PV_0` (the $t=0$ value), and
2. optionally a **profile**: signed per-path PV at a set of future dates, $\{PV_k(\omega)\}$, from which exposure measures (EPE, ENE, PFE) are computed.

The hard part is **early exercise / path-dependent decisions**. Whenever the product contains a `decide/american` event (Bermudan/American optimal stopping, callable/puttable, autocall), the value at that date depends on the *continuation value* — the conditional expectation of future cashflow given the current state — which is not known pathwise. The engine estimates it by **Longstaff-Schwartz (LSM) regression**: regress realized future cashflow on a basis of functions of the current state, and use the fitted surface as the continuation value.

The engine is a **two-pass** estimator:

- **Pass A (fit)**: simulate paths, walk **backward** through events, and at each decision date fit a regression of discounted future cashflow on the state basis. This yields a *frozen exercise policy* (the regression coefficients). The Pass-A price is in-sample and biased (high for a holder's `max`).
- **Pass B (evaluate)**: simulate **fresh** paths, walk **forward** applying the frozen policy from Pass A. This gives an out-of-sample, low-bias price and the per-path PV used for profiles.

Everything below specifies the mathematics of each stage and every configurable option: basis families, regularization (ridge / SVD / LASSO), the protected (intrinsic) column, stratification, rate-curve PCA, auto-regressor inference, and the taxonomy-driven configuration precedence.

2. Notation and the discrete time grid

- Paths are indexed by $\omega = 1..N$; events by $i = 0..M$ at increasing dates $t_0 < t_1 < \dots < t_M$. Same-date events are ordered by a (date, seq) tuple.
- $r_s(\omega)$ is the simulated short rate; the **pathwise discount factor** between two event dates is

$$D_{i,j}(\omega) = \exp\left(-\int_{t_i}^{t_j} r_s(\omega) ds\right) \approx \prod_{i < k \leq j} e^{-r_{k-1}(\omega) \Delta t_k}$$

computed once per pass by vectorised trapezoidal/cumsum integration of the short-rate path into the bank account $B(t) = \exp(\int_0^t r)$. - $cf_j(\omega)$ is the cashflow emitted at event j (a pay event's amount, signed: positive to the holder; discounted events carry D , pay_undisc events do not). - $g_i(\omega)$ is the **immediate (intrinsic) value** at a decision event i — the payoff if the holder stops/exercises now. - $X_i(\omega) \in \mathbb{R}^d$ is the **regressor vector**: the values of the state variables flagged as regressors (and, optionally, auto-appended spot prices) at event i . - $alive_i(\omega)$, $forced_i(\omega)$ are per-path boolean masks (knock-out / mandatory stop status carried in state).

3. The Longstaff-Schwartz backward recursion (Pass A)

The exact dynamic program is the Bellman recursion for the value V_i :

$$V_i(\omega) = \max(g_i(\omega), C_i(\omega)), \quad C_i(\omega) = \mathbb{E}^Q[D_{i,i+1} V_{i+1} | \mathcal{F}_{t_i}]$$

where C_i is the continuation value — the conditional expectation of the discounted value of continuing. The terminal condition is $V_M = g_M$ (or the final cashflow). C_i is not known pathwise, so the engine replaces it by a regression estimate.

3.1 Cashflow carry. Walking backward from $i = M$ to 0 , the engine maintains a per-path **future-cashflow accumulator** $Y_i(\omega)$ — the discounted sum of all cashflows that occur strictly after the current decision, *under the exercise policy decided so far*:

$$Y_i(\omega) = \sum_{j>i} D_{i,j}(\omega) cf_j(\omega)$$

At each non-decision event the accumulator simply absorbs that event's discounted cashflow: $Y \leftarrow Y + D \cdot cf$. (A fix event updates state but emits no cashflow.)

3.2 The regression. At a decision event i , the continuation value is approximated by a linear model in a feature map Φ of the (standardised) state:

$$C_i(\omega) \approx \hat{C}_i(\omega) = \beta_i^\top \Phi(X_i(\omega))$$

The coefficients are the least-squares fit of the carried future cashflow Y_i (the regression **target**) on the basis, over the **active sample** $\mathcal{A}_i$ (the paths eligible for the decision — see §3.4):

$$\hat{\beta}_i = \arg \min_{\beta} \sum_{\omega \in \mathcal{A}_i} (Y_i(\omega) - \beta^\top \Phi(X_i(\omega)))^2$$

The fitted surface $\hat{C}_i(\omega) = \hat{\beta}^\top \Phi(X_i(\omega))$ is the per-path continuation value.

3.3 The exercise decision. With the continuation estimate in hand, the engine applies the **decision rule**, which depends on the event's **polarity**:

- **Holder optimises** (**polarity** = "max" — American/Bermudan, putable, autocall holder). Exercise where the immediate value beats continuation, gated on being in-the-money so the regression is not extrapolated into deep-OTM noise:

$$\text{stop}_i = \text{forced}_i \vee (\text{alive}_i \wedge (g_i > 0) \wedge (\hat{g}_i > \hat{C}_i))$$

- **Issuer optimises** (**polarity** = "min" — callable bond). The issuer calls (stops) where continuing is *more expensive* than the immediate call value:

$$\text{stop}_i = \text{forced}_i \vee (\text{alive}_i \wedge (\hat{g}_i < \hat{C}_i))$$

A **forced** condition (e.g. an autocall barrier or a mandatory redemption) stops the path regardless of optimality. Where a path stops, its future-cashflow accumulator is overwritten by the immediate value:

$$Y_i \leftarrow \text{stop}_i ? g_i : Y_i \text{ (discounted to } t_i).$$

Note the asymmetry the engine encodes precisely: for **max** the comparison uses the **discounted** immediate value against $\hat{C}_i$ *and* requires $g_i > 0$; for **min** there is no positivity gate (the issuer calls wherever feasible).

3.4 The active / fit sample. Two filters (config flags) shape $\mathcal{A}_i$:

- **use_only_alive** (default **True**): restrict the regression to paths that are still alive at t_i (not knocked out), since dead paths carry no future decision.
- **use_only_itm** (default **False**, the classic LSM convention when set True): restrict the *fit* to in-the-money paths ($g_i > 0$). This improves conditioning in deep-OTM regions. It is only meaningful for **polarity** = "max"; for "min" it is left off.

Restricting the **fit** sample does not restrict the **decision**: the fitted surface is still applied to all alive paths.

3.5 The Pass-A price. After the backward walk reaches t_0 , the accumulator Y_0 is the per-path discounted value under the (in-sample) policy, and

$$\widehat{PV}_0 = \frac{1}{N} \sum_{\omega=1}^N Y_0(\omega)$$

is the Pass-A price — in-sample, hence biased (high for **max**, low for **min**). The frozen coefficients $\{\hat{\beta}\}$ constitute the **exercise policy**.

4. Pass B — unbiased evaluation and the PV grid

Pass A produces an exercise *policy*; evaluating that policy **on the same paths it was fit on** is optimistically biased. Pass B removes that bias:

1. Simulate a **fresh, independent** path set (different RNG seed).

2. Walk **forward** through events. At each decision event, evaluate the frozen $\hat{C}_i = \beta^T \Phi(X_i)$ and apply the **same** decision rule (§3.3) — but never refit. The policy is fixed.
3. Accumulate realized discounted cashflow per path forward to t_0 .

The mean over Pass-B paths is the reported **price**; its sample standard error is the Monte-Carlo error. Pass A's price is also returned as a diagnostic (the gap $PV_A - PV_B$ is the regression/early-exercise bias).

4.1 The PV profile (exposure grid). For counterparty-credit / XVA work the engine produces signed per-path PV at a set of **profile dates** $\{t_k\}$. At each profile date the value of the deal *seen from* t_k is the discounted remaining realized cashflow plus, if the deal is still alive, the continuation value:

$$\widehat{PV}_k(\omega) = \sum_{j: t_j > t_k} D_{k,j}(\omega) cf_j(\omega) + \mathbb{1}[\text{alive}_k] \hat{C}_k(\omega)$$

This reuses the frozen regression surfaces $\hat{C}_k$ from Pass A (interpolated/keyed to the nearest decision date) so that the early-exercise optionality is correctly valued at every profile date, not just at t_0 . The result is the **grid** $\{PV_k(\omega)\}$ — N paths $\times$ K profile dates — that drives all exposure metrics.

4.2 Exposure measures. From the PV grid:

$$EPE(t_k) = \mathbb{E}^Q[(PV_k)^+], \quad ENE(t_k) = \mathbb{E}^Q[(PV_k)^-]$$

$$PFE_\alpha(t_k) = \inf \{x : \mathbb{P}(PV_k \leq x) \geq \alpha\}$$

EPE/ENE are the expected positive/negative exposures (the building blocks of CVA/DVA); PFE is the α -quantile of exposure (a tail risk measure). These are pathwise reductions of the grid and require no further model machinery.

5. The feature map Φ — basis families

The regressor vector $X_i \in \mathbb{R}^d$ is first **standardised** column-wise on the fit sample (zero mean, unit variance), which is essential for conditioning and so that ridge/centres are scale-free:

$$\tilde{X}^{(d)} = \frac{X^{(d)} - \mu_d}{\sigma_d}$$

The feature map Φ then expands the standardised inputs. Three families are available (RegressionConfig.basis), all sharing the convention that a **constant column** is prepended separately so it is never shrunk by regularization.

5.1 Total-degree polynomial ("polynomial", default). All monomials in the standardised inputs up to total degree $p = \text{degree}$ (cross terms included), excluding the constant:

$$\Phi_{poly}(\tilde{X}) = \left\{ \prod_d \tilde{X}_d^{a_d} : \sum_d a_d \leq p, \mathbf{a} \neq \mathbf{0} \right\}$$

Fast and simple; **ill-conditioned at high degree** (the monomial Gram matrix becomes near-singular for $p \geq 4$). The default `degree = 2` is the standard LSM choice.

5.2 Probabilists' Hermite ("hermite"). Same total-degree multi-index structure, but each axis uses Hermite polynomials instead of raw monomials:

$$He_0 = 1, He_1 = x, He_{n+1}(x) = x He_n(x) - n He_{n-1}(x)$$

Because the He_n are orthogonal under the Gaussian weight, the basis is **dramatically better conditioned** than monomials at the same degree — the preferred choice when high-degree LSM is needed on centred/scaled state.

5.3 Gaussian radial basis functions ("rbf" / "gaussian_rbf"). `n_centers` kernels placed at quantiles (1-D) or by k-means (≥ 2 -D), each

$$\phi_k(\tilde{X}) = \exp\left(-\frac{\|\tilde{X} - c_k\|^2}{2h^2}\right)$$

with bandwidth `h` either supplied (`rbf_bandwidth`) or auto by Silverman's rule scaled by `bandwidth_scale` (default 2.0):

$$h = \text{scale} \cdot \left(\frac{4}{(d+2)N}\right)^{1/(d+4)} \hat{\sigma}$$

RBFs **adapt locally** and do not oscillate near kinks/barriers, so they are robust for barrier options and autocallables where polynomial fits ring. The number of basis functions grows linearly in `n_centers`, avoiding the combinatorial blow-up of high-degree polynomials in high dimension.

6. Regularization — the solve

Given the design matrix Φ (constant + basis columns) and target Y , the coefficients are solved by a regularized least-squares. The mode is selected by `RegularizationConfig.mode`, or — for backward compatibility — by the legacy `ridge` / `svd_threshold` fields:

- `ridge > 0` → **RIDGE** mode (Tikhonov), strength `ridge`;
- `ridge == 0` → **SVD** mode, truncation threshold `svd_threshold`.

When an explicit `RegularizationConfig` is supplied it takes precedence and the two are **mutually exclusive**.

6.1 Ridge (Tikhonov). Shrink all (non-constant) coefficients toward zero:

$$\hat{\beta} = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top Y$$

Stable and the default for ill-conditioned bases. Implemented via the SVD form

$$\hat{\beta} = \sum_k \frac{s_k}{s_k^2 + \lambda} (u_k^\top Y) v_k$$

which is numerically robust (no explicit normal-equations inverse).

6.2 SVD truncation. Decompose $\Phi = U S V^\top$ and **drop** singular directions whose relative singular value falls below `$\tau = \text{svd\_threshold}$` :

$$\hat{\beta} = \sum_{s_k/s_{\max} \geq \tau} \frac{u_k^\top Y}{s_k} v_k, \quad \Phi = U S V^\top$$

This removes near-null-space directions entirely rather than shrinking them, giving a minimum-norm solution on the retained subspace. Diagnostics (condition number, retained rank) are recorded per fit.

6.3 LASSO / Elastic-Net (optional `mode = "lasso" / "elastic"`). For feature-selective shrinkage — driving small coefficients to *exactly* zero rather than uniformly down:

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{2} \|Y - \Phi\beta\|_2^2 + \alpha \|\beta\|_1$$

Elastic-net adds an `l2` term with mixing `l1_ratio`. Useful when many basis columns are spurious (the cross-measure P-vs-Q setting). Solved by coordinate descent on the residualized system (§6.4).

6.4 The protected (intrinsic) column. A subtle but important detail: the engine supports a **protected block** `C` of columns that must keep their natural OLS coefficient and **never** be shrunk — typically the constant and an intrinsic-value column (the “auto-intrinsic regressor”). The fit residualizes the free basis against the protected block before applying shrinkage:

$$\Phi = [C \mid \Phi_{free}], \quad \Phi_{res} = \Phi_{free} - C(C^\top C)^{-1}C^\top \Phi_{fre}$$

i.e. (1) OLS-fit `Y` on `C`; (2) project the free basis `Φ_free` onto the orthogonal complement of `C`; (3) apply ridge/SVD/LASSO only to the residualized `Φ_res` against the residualized target. This guarantees the intrinsic value is represented exactly while the basis only explains the excess continuation value — which markedly stabilises the early-exercise boundary.

7. Stratification (bucketed regression)

A single global regression can be a poor fit when the continuation surface has sharply different regimes (e.g. above vs below a barrier). **Stratification** (`RegressionConfig.bucketing`) partitions the paths into `B` buckets and fits an **independent** regression per bucket:

$$\hat{C}_i(\omega) = \sum_{b=1}^B \mathbb{1}[b(\omega) = b] \beta_{i,b}^\top \Phi(X_i(\omega))$$

where `b(ω)` is the bucket assignment. Two methods:

- “kmeans” — cluster paths in a chosen subset of state features (`feature_idx`), minimising within-cluster variance:

$$\{c_b\} = \arg \min_{\omega} \sum_b \min \|X^{strat}(\omega) - c_b\|^2$$

- “quantile” — 1-D quantile bins (edges frozen after fit).

Critical invariant for reprice stability: the cluster centres / bin edges are **frozen after the fit pass** and reused for every subsequent bump/reprice, so a finite-difference Greek does not see paths hop between buckets (which would inject discontinuity noise into the

sensitivity). Each bucket gets its own basis fit; a path's prediction uses only its own bucket's coefficients (a `predict_mask` zeroes the others).

8. Rate-curve PCA (dimensionality reduction of rate regressors)

For interest-rate products the natural state is the whole simulated yield curve — too high-dimensional to regress on directly. **PCA** (`RatePcaConfig` / `PcaSpec`) reduces it to the few dominant modes (level / slope / curvature).

Let $\ell(\omega)$ be the vector of log-discount-factors (or log forward rates) across tenors at the regression date, demeaned across paths. The covariance and its eigendecomposition are computed on the **Q paths** (fit measure):

$$\Sigma = \frac{1}{N} \sum_{\omega} (\ell(\omega) - \bar{\ell})(\ell(\omega) - \bar{\ell})^T$$

$$z_m(\omega) = w_m^T (\ell(\omega) - \bar{\ell}), \quad \Sigma w_m = \lambda_m w_m, \quad \lambda_1 \geq \lambda_2 \geq \dots$$

The eigenvectors `w_m` (sorted by eigenvalue) are the principal modes; the **scores** $z_m(\omega) = w_m^T (\ell(\omega) - \bar{\ell})$ become the rate regressors fed to the LSM basis. `n_components` $\in \{1, 2, 3\}$ (capped at 3 — beyond that, low-variance components add noise more than signal, especially in cross-measure P-vs-Q work).

Adaptive truncation. When `min_variance_share = v_min` is set, components whose explained-variance ratio falls below the threshold are dropped, keeping at least one:

$$\text{EVR}_m = \frac{\lambda_m}{\sum_k \lambda_k}, \quad \text{keep } m : \text{EVR}_m \geq v_{\min}$$

PCA reduces a curve of, say, 20 tenor points to 2-3 orthogonal regressors that capture essentially all of the curve's variation — keeping the LSM basis low-dimensional and well-conditioned.

9. Auto-regressor inference

The LSM regression is only as good as its regressors. If the script author forgot to flag a path-dependent state variable as a `regressor`, the continuation surface will be blind to it. The engine has a **compile-time inference pass** (`regressor_inference`, `compile_script(mode=...)`) that detects likely regressors. Modes: `"off"` (default, silent), `"warn"` (detect + warn), `"auto"` (detect + silently flag).

Layer 2 — accumulator & gate detection. Scanning the (AST of the) payoff and `do/set` expressions, a state variable is inferred to be a regressor if it is:

1. the **label of an accumulator** function — the first positional argument of any `RUNNING_*`, `SUP/INF`, `SNAPSHOT`, `PERF_SUM/RATIO_*` family call. Accumulators are path-dependent by construction, hence almost always regression-relevant; or
2. a variable that participates in a **binary gate** pattern (a comparison feeding a `where/ternary` that drives a cashflow), i.e. it shapes the exercise region.

DSL builtins are excluded (only genuine state names are flagged). The pass mutates the state spec in place under "auto", flipping regressor from False to True.

Layer 3 — alive-mask gaps (warning only). Detects state used in an alive/ survival mask that is not a regressor, which can bias the exercise boundary; emits a warning but does not auto-flag (the fix is usually a modelling choice).

Spot auto-append. Independently of inference, at each decision the engine appends the current **spot prices** of the underlyings to X_i (unless they are already present via an explicit regressor), so the continuation value always sees the primary risk factors. Explicit `state_regressors` on an event override this.

10. Taxonomy and the configuration-precedence chain

Every regression knob above (basis, regularization, PCA, rate-regressors) can be set globally, per product type, per product, or per deal. The resolver walks a **4-level precedence chain** (highest → lowest), per config key:

1. `deal.metadata[<key>]` — per-deal override;
2. `deal.product.descriptor.parameters[<key>]` — per-product attachment;
3. `recommender.recommend(product.taxonomy)` — per product **type**;
4. `policy.<config>` — global default.

A `provenance` dict records which level supplied each resolved config ('deal-override', 'product-descriptor', 'recommender:<name>', 'default'), for audit/replay.

Taxonomy-driven recommenders. A product carries a `ProductTaxonomy` (SIMM- aligned: dominant product class, the risk-class → risk-metric map, buckets). A `Recommender` maps that taxonomy to a recommended config — e.g. an autocallable (barrier, kinky payoff) is steered toward an **RBF basis**; a high-dimensional rate Bermudan toward **PCA + ridge**; a smooth vanilla toward **low-degree polynomial**. This is how a freshly compiled, un-annotated product still gets a sensible regression setup automatically.

11. End-to-end algorithm (pseudocode)

```
PRICE(product, model, regression_cfgs, profile_dates=None,
       n_paths, seed_a, seed_b):

    # ---- resolve per-event regression config (taxonomy precedence) ----
    for each decision event i:
        cfg[i] = resolve_config(deal.metadata, product.descriptor,
                               recommender(product.taxonomy), policy)
        # cfg carries: basis (+degree/n_centers), regularization mode (+λ/τ),
        #             bucketing, rate-PCA, use_only_alive/itm, regressors

    # ===== PASS A : fit the exercise policy =====
    paths_A = model.simulate(n_paths, seed_a)      # under Q
    B(t)     = exp(cumsum(r·Δt))                   # bank account, per path
    cf_future = zeros(N)
    for i = M down to 0:                          # BACKWARD walk
        ev = product.events[i]
        if ev is a cashflow event:
            cf_future += discount(ev.date)·ev.amount(paths_A, state_i)
        continue
```



```

    if ev is a decision event:
        X, controls = build_features(paths_A, state_i, ev.regressors) # +spots
        X_std      = standardize(X)                                #  $\mu, \sigma$  on fit sample
        alive      = alive_mask [  $\wedge$  ITM mask if use_only_itm ]
        if rate-PCA enabled: replace rate cols of X_std with PCA scores z_m
        if bucketing: assign buckets b( $\omega$ ) (freeze centres)
         $\beta[i]$       = fit_block( $\Phi(X\_std[\cdot])$ ), controls, cf_future[alive], cfg[i])
                        # protected constant/intrinsic + ridge|SVD|LASSO
         $\hat{C}$           =  $\beta[i] \cdot \Phi(X\_std)$                         # continuation estimate
         $\hat{g}$           = discount(ev.date)  $\cdot$  g_i                    # discounted immediate
        if polarity == "min": exercise = alive  $\wedge$  ( $\hat{g} < \hat{C}$ )
        else:         exercise = alive  $\wedge$  (g_i>0)  $\wedge$  ( $\hat{g} > \hat{C}$ )
        stop          = forced v exercise
        cf_future     = where(stop,  $\hat{g}$ , cf_future)
    price_A = mean(cf_future)                                     # in-sample (biased)

# ===== PASS B : evaluate out-of-sample =====
paths_B   = model.simulate(n_paths, seed_b)                    # FRESH paths
cf_fwd    = zeros(N); alive = ones(N)
for i = 0 .. M:                                                # FORWARD walk
    ev = product.events[i]
    if ev is a cashflow event: cf_fwd += discount  $\cdot$  amount; continue
    if ev is a decision event:
        X_std = standardize_with_frozen_ $\mu\sigma$ (features(paths_B, state_i))
         $\hat{C}$      =  $\beta[i] \cdot \Phi(X\_std)$                         # FROZEN policy, no refit
        apply same decision rule  $\rightarrow$  update alive / record stop & payoff
    price   = mean(cf_fwd); stderr = std(cf_fwd)/sqrt(N)

# ===== optional : PV PROFILE grid =====
if profile_dates:
    for k, t_k in profile_dates:
        PV[:,k] =  $\sum_{t_j > t_k} \text{discount}(t_k, t_j) \cdot \text{cf}_j$       # realized tail
                  + alive_k  $\cdot \hat{C}_k(\text{state at } t_k)$            # frozen cont. value
        EPE[k]=mean(PV[:,k]^+); ENE[k]=mean(PV[:,k]^-)
        PFE_ $\alpha$ [k]=quantile(PV[:,k],  $\alpha$ )

return price, stderr, price_A, (PV grid, EPE, ENE, PFE if requested)

```

12. Configuration reference (every knob)

Config object	Field	Meaning / values
RegressionConfig	basis	"polynomial" (default) $\cdot$ "hermite" $\cdot$ "rbf" / "gaussian_rbf" $\cdot$ or a basis instance
	degree	total-degree cap p for polynomial/hermite (default 2)
	n_centers	number of RBF kernels (default 16)
	rbf_bandwidth	absolute RBF scale; None $\Rightarrow$ Silverman $\times$ bandwidth_scale
	regularization	a RegularizationConfig (takes precedence over legacy fields)
	ridge	legacy Tikhonov strength; $>0 \Rightarrow$ RIDGE mode
	svd_threshold	legacy SVD relative cut τ ; used when ridge==0
	bucketing	a Bucketing (stratification)

Config object	Field	Meaning / values
	<code>use_only_alive</code>	restrict fit to alive paths (default True)
	<code>use_only_itm</code>	restrict fit to in-the-money paths (default False; classic LSM when True; max only)
RegularizationConfig	<code>mode</code>	RIDGE · SVD · LASSO · ELASTIC
	<code>ridge</code>	Tikhonov λ (default 1e-6 in explicit RIDGE)
	<code>svd_threshold</code>	SVD cut τ (default 1e-8)
	<code>l1_ratio</code>	elastic-net l_1/l_2 mix
Bucketing	<code>method</code>	"none" · "kmeans" · "quantile"
	<code>k</code>	number of buckets B
	<code>feature_idx</code>	which X columns to cluster on
RatePcaConfig / PcaSpec	<code>n_components</code>	1, 2, or 3 (level/slope/curvature; capped at 3)
	<code>min_variance_share</code>	adaptive truncation threshold $v_{\min}$
<code>compile_script</code>	<code>mode</code>	regressor inference: "off" · "warn" · "auto"
(per event)	<code>polarity</code>	"max" (holder) · "min" (issuer)
	<code>forced_stop</code>	mandatory-stop condition
	<code>state_regressors</code>	explicit regressor list (overrides spot auto-append)
PricingResult	<code>price</code>	Pass-B (out-of-sample) price
	<code>pass_a_price</code>	Pass-A (in-sample, biased) diagnostic
	<code>stderr</code>	Monte-Carlo standard error

13. Implementation checklist

To reproduce the PV grid from scratch you need, in order:

1. **A factor model** that simulates Q paths of all underlyings + the short rate on the event date grid, and from which the pathwise bank account $B(t)$ (hence discount factors $D_{\{i,j\}}$) is obtained by `cumsum` of $r \cdot \Delta t$.
2. **The compiled product**: ordered `(date, seq)` events with, per event, an `amount/payoff` evaluator over `(snapshot, state)`, a `discounted` flag, and — on decisions — `polarity`, `forced_stop`, and the regressor list. (This is the output of `compile_amc`; see the DSL reference.)
3. **State evolution**: apply each `fix/set` event's state update in `(date, seq)` order; carry per-path state vectors forward.
4. **The regression kernel** (§5–§6): `standardise` → `basis expand` → `protected-block residualize` → `ridge/SVD/LASSO solve` → `predict`. With optional bucketing (§7) and rate-PCA (§8) preprocessing of X .
5. **The two-pass driver** (§3–§4, §11): backward fit on paths A, forward evaluate on paths B, decision rule by polarity.
6. **Profile reduction** (§4.1–§4.2): per-profile-date discounted realized tail + frozen continuation value, then EPE/ENE/PFE.

The DSL reference specifies item 2 (the product); this document specifies items 1 and 3-6 (the engine). Together they define the full path from `.amc` text to the PV grid.

14. The analytic credit / tranche pricing kernel

Sections 1-13 specify the **Longstaff-Schwartz regression** engine. Synthetic CDO tranches are priced by a separate **analytic** kernel (`amc_engine/models/credit_pkg/tranche_pricer.py`): a one-factor Gaussian copula with a base-correlation surface. Marginals (per-issuer forward default probabilities) and dependence (the copula `rho`) are kept strictly separate; the copula `rho` is **not** the same object as a simulation model's factor loading.

14.1 Tranche expected loss

For attach `a`, detach `d`, with renormalised strikes `a_t`, `d_t`, `width = d_t - a_t`:

$$EL = ((d_t - P_d) - (a_t - P_a)) / width, \quad P_x = E[(x - L)^+]$$

(`width = 0`, i.e. realised loss $\geq$ detach, $\Rightarrow$ fallback `EL = 1.0`). The put values `P_a`, `P_d` come from an **Andersen-Sidenius-Basu bucket recursion** over the pool loss `L`, integrated over the common Gaussian factor by Gauss-Hermite (`gh_nodes`). Conditional on the factor the names are independent, so each name's loss is convolved into a `grid`-bucket distribution (NumPy and numba kernels agree).

14.2 Recovery models — the LGD-bucket rule

A defaulting name loses `(1 - R) · weight` (its loss-given-default) with **default probability** `p`. The 2-state path therefore shifts the loss bucket by the LGD-sized amount `(1 - R) · w̃` with probability `p` — *not* the full weight `w̃` with probability `p · (1 - R)`. Those share a mean but differ in distribution, and a tranche is non-linear in loss, so the latter biases the EL. At `rho → 0` the corrected kernel matches the exact independent enumeration $\sum_k C(N, k) p^k (1-p)^{N-k} \text{clip}((LGD \cdot k/N - a)/(d-a), 0, 1)$ to machine precision (ratio 1.00 at `N = 5, 25`; verified in `tools/verify_engine_docs.py`). The 3-state path adds stochastic recovery (`recovery_var_frac`); with variance 0 it collapses to the deterministic result.

`el_mode='fast'` runs one recursion at maturity and interpolates $EL(a, t) = EL_{mat} \cdot \bar{L}(a, t) / \bar{L}(a, T_{mat})$ along a loss clock (one recursion per profile date).

14.3 Forward mark-to-market sweep

`tranche_pv_paths(smd, profile_dates, *, index, attach, detach, spread, schedule, cfg, currency, effective_date, base_corr_models=None, upfront=None, is_upfront_payer=True, nominal_exchange=False, amortizing=None)` returns the per-unit protection-buyer value on each path/date. Four correctness rules govern it:

1. **Skip fully-elapsed periods** ($t \leq a$) — otherwise `|MtM|` grows toward maturity instead of running off.
2. **Scale by tranche outstanding, not pool survivor**: `ON_tr = clip((detach - L_real) / (detach - attach), 0, 1)` with `L_real = (1 - alive) · (w · (1 - R))`, applied to both legs; a wiped tranche must not re-pay settled protection.
3. **Dirty premium** for exposure (full coupon period $t_1 - t_0$) — produces the intra-period sawtooth (EE ramps within a coupon period, drops $\approx$ one coupon at each coupon date).
4. **(date, seq) event ordering** (no float-epsilon nudges).

Measure is set by which paths feed `SimulatedMarketData`: `forward_survival(issuer,t,T)` is driven entirely by the path-carried hazard state, so the same kernel on P-paths gives the P-measure forward MtM (used by the closed-form CCR route — Execution Spec §9).